

Meditation Consolidation: Agent Harness Orchestration Patterns

Overview

Three branches explored the orchestration of multi-agent systems through complementary lenses: how agents share data (state coordination), what happens when things fail (resilience), and how the system stays bounded (resource governance). Across 39 exploration files spanning 3 recursion levels, a unified architectural model emerged — one that is already partially implemented in the CRUX-Compress codebase and points toward concrete improvements.

Branch 1: State Coordination and Handoff Mechanisms

Key Discoveries

State coordination operates as a **three-layer protocol stack**:

Layer	Concern	Mechanisms
Transport	Where data moves	File-based polling, predictable paths, existence as signal
Propagation	How intent flows	<code>alwaysApply</code> rules, spawn-time args, config pull
Serialization	What is exchanged	Frontmatter contracts, four-format stack (JSON → YAML → Markdown → CRUX)

A **push-pull duality** unifies all three layers: pull (agents independently read shared state from known locations) for persistent defaults, push (explicit parameters at spawn time) for ephemeral overrides. This mirrors the plugin system's advisory-gate-with-opt-out architecture.

The system consistently chooses **explicit over implicit** (source fields, `alwaysApply` rules, frontmatter contracts over inference from paths or timestamps), **repair over prevention** (eventual consistency with REM sleep rather than strict referential

integrity), and **consumer-optimized formats** (LLM comprehension as the primary design driver).

Central Challenge: The Knows-vs-Acts Gap

Delivering state to an agent is necessary but not sufficient for behavioral compliance. This manifests as: - Detection without verification in file protocols - Loaded but ignored rules in session propagation - Defined but violated contracts in serialization

The proposed **compliance testing pyramid** — directive trust (weakest) → self-report → output-pattern analysis → side-effect observation → adversarial verification (strongest) — provides a framework for systematically closing this gap.

Branch 2: Failure Handling and Resilience

Key Discoveries

The harness implements a **four-layer defence-in-depth stack**:

Layer	Function	Example
Demand reduction	Eliminate unnecessary work	Shared-agent-runs (N calls → 1), skip-by-default
Concurrency limiting	Reduce peak pressure	maxForks: 2, dependency-based phasing
Reactive recovery	Recover from transient failures	Exponential backoff + jitter (BASE=2s, MAX=60s, 5 retries)
Hard backstop	Kill runaway processes	Wall-clock deadline (60min default)

The deepest epistemological finding: **an agent's narrative is a hypothesis, the filesystem is ground truth**. Adversarial verification catches real failures at a 25% hit rate, with variance correlating to integration complexity rather than task count.

Three Critical Gaps

1. **Semantic hang detection**: Five undetected hang patterns (tool-call loops, output stagnation, thinking-without-acting, file-polling deadlocks, phantom completions). A **progress token framework** — tracking `lastProgressTimestamp` based on novel actions — would unify all five into a single configurable mechanism.

2. **Partial-result degradation:** Current all-or-nothing architecture wastes successful work. For exploratory operations (meditation, recall), graceful consolidation with gap annotation is strictly better. For transactional operations (spec execution, memory creation), correlation-first triage should distinguish systemic from isolated failures.
 3. **Graduated verification:** Uniform adversarial verification is wasteful. A risk-scoring heuristic (file creation +3, cross-cutting registration +3, documentation with paths +2, first-time agent +2) should drive tiered verification intensity.
-

Branch 3: Resource Governance and Bounded Execution

Key Discoveries

Resource governance follows a **three-layer control architecture** analogous to TCP congestion control:

Layer	Function	Agent Equivalent	TCP Analogue
Demand reduction	Eliminate unnecessary work	Batching, skip-by-default, specialization	Connection reuse
Admission control	Bound concurrent execution	Static caps, phase parallelism, deadlines	Congestion window
Reactive recovery	Handle residual transients	Exponential backoff, jitter, retry budgets	Fast retransmit

Three unifying principles: **reduce demand before increasing tolerance** (eliminating work compounds savings), **static ceilings with negotiable compliance paths** (maximum cost is predictable, path to compliance is flexible), and **human escalation as universal terminal** (agents refuse to make irreversible decisions).

Central Insight: Cost and Blast Radius Are Orthogonal

A 13-agent recursive meditation is expensive but safe (read-only). A single-agent destructive operation is cheap but dangerous. Both dimensions must independently escalate governance tier classification. The meditate system's read-only-with-opt-in-persistence pattern demonstrates that bounding blast radius is a governance concern distinct from bounding cost.

The Missing Middle Layer

The codebase has strong implementations of layers 1 (demand reduction) and 3 (reactive recovery) but nearly no layer-2 admission control beyond static caps. Missing: dynamic concurrency controllers, admission queuing, staggered starts, correlated failure detection, and graceful shutdown signals for hard deadlines.

Cross-Branch Connections

1. The Defence-in-Depth Stack Is Universal

All three branches independently converged on layered architectures with the same ordering: **eliminate** → **bound** → **recover** → **escalate**. This isn't coincidence — it reflects the fundamental principle that reducing demand has compounding returns (fewer calls = lower cost AND fewer failures AND less state to coordinate), while increasing tolerance has diminishing returns.

2. The Push-Pull Duality Spans All Concerns

Concern	Pull (shared state)	Push (explicit params)
Coordination	Config files, alwaysApply rules	Spawn-time arguments
Resilience	Filesystem ground truth	Error context propagation
Governance	Static caps in config	Session-scope flag inheritance

The mature pattern is hybrid: pull for defaults, push for overrides. This holds across all three domains.

3. File-Based Coordination Enables Partial-Result Recovery

The meditation protocol's file-based coordination is simultaneously a coordination mechanism (Branch 1), a resilience enabler (Branch 2), and a governance primitive (Branch 3): - **Coordination**: Predictable paths enable decoupled agents to find each other's outputs - **Resilience**: Persistent child outputs survive parent crashes, enabling selective retry at the lowest level - **Governance**: Read-only exploration bounds blast radius; file existence gates persistence decisions

4. The Knows-vs-Acts Gap and Trust Inversion Are Two Faces of One Problem

Branch 1's "knows-vs-acts gap" (delivering state $\neq$ compliance) and Branch 2's "trust inversion" (agent narratives are hypotheses, filesystem is ground truth) describe the same fundamental challenge: **LLM agents are probabilistic executors operating in deterministic environments**. The system must bridge this gap at every layer — through redundant constraint expression, adversarial verification, and ground-truth observation rather than self-report.

5. Escalation Ladders Are Composable Primitives

Branch 3's formal escalation ladder (initial $\rightarrow$ step $\rightarrow$ ceiling $\rightarrow$ invariant $\rightarrow$ terminal) appears across all branches: - **Coordination**: Schema evolution is an additive escalation ladder (optional $\rightarrow$ required $\rightarrow$ canonical) - **Resilience**: Retry strategies are time-domain escalation ladders (2s $\rightarrow$ 4s $\rightarrow$ 8s $\rightarrow$ 16s $\rightarrow$ 32s $\rightarrow$ escalate) - **Governance**: Compression targets are space-domain escalation ladders (33% $\rightarrow$ aggressive $\rightarrow$ maximum $\rightarrow$ manual review)

This five-parameter interface (initial level, step function, ceiling, invariant predicate, terminal action) could be abstracted into a reusable primitive.

Potential Directions for Further Exploration

1. **Progress Token Framework**: Design and prototype a `lastProgressTimestamp` tracker that detects all five semantic hang patterns through a single mechanism. This is the highest-value concrete improvement — a single hung leaf currently blocks entire meditation trees indefinitely.
2. **Dynamic Admission Control**: Prototype a TCP-inspired admission controller that adjusts concurrency based on rate-limit signal frequency. The static-cap gap is the largest architectural hole across all three branches.
3. **Compliance Testing Infrastructure**: Move critical coordination checks from levels 1-3 (directive trust, self-report, output patterns) to levels 4-5 (side-effect observation, adversarial verification) of the compliance pyramid. Start with file-write verification — the most concrete and frequently-failed check.
4. **Escalation Ladder Abstraction**: Extract the five-parameter escalation ladder into a reusable skill or utility, replacing the scattered implementations in compression, retry, and type demotion.

5. **Cost-Blast-Radius Matrix:** Formalize the two-axis governance classification so every agent operation is tagged with both its cost tier and its blast radius tier, enabling independent gating.
-

Actionable Insights

Highest Priority (Concrete Bugs / Gaps)

- **Add polling timeout to meditate file-based coordination** — a single hung depth-3 agent currently blocks the entire tree forever
- **Widen jitter from $\pm 15\%$ to $\pm 50\%$** (or decorrelated jitter) to safely support higher parallelism without thundering-herd collisions
- **Verify file writes after creation** — the Write tool can silently fail; `1s` after every critical write is cheap insurance

Medium Priority (Architectural Improvements)

- **Implement graceful partial-result consolidation** for exploratory operations — proceed with N-1 results and annotate the gap, rather than blocking on all N
- **Centralize path conventions** — five file-based coordination families with different naming conventions are specified in prose; a shared path-computation utility would prevent drift
- **Add graduated adversarial verification** with risk scoring to optimize the 25% hit rate against verification cost

Strategic (Design Patterns to Adopt)

- **Reduce demand before increasing tolerance** — this principle should be the default response to any resource pressure, before tuning retries or raising limits
- **Separate cost governance from blast-radius governance** — read-only exploration with opt-in persistence is the model for bounded-risk agent operations
- **Use retry frequency as a diagnostic signal** — if retries fire often, the proactive settings (not the retry parameters) need adjustment